Nesne Yönelimli Programlama

Çıkmış Vize Soruları ve Ayrıntılı Çözümleri

Sorular: sınav fotoğrafları, docx/md dosyaları, WhatsApp aktarımları
Çözümler: slaytlar, ders kitabı ve hocanın derste anlattıkları

9 Kasım 2026

Bu dosya hakkında

- Klasördeki bütün kaynaklarda bulunan **vize** sorularını içerir. Kaynak ve güvenilirlik her sınavın başında belirtilmiştir: *kâğıt fotoğrafı*, *docx/md*, *öğrenci aktarımı*. Final soruları (design pattern, SOLID kodlama) vize kapsamında olmadığı için alınmadı; yalnızca vize konularıyla örtüşenler alıştırmalarda kullanıldı.
- Vize kapsamı: **Chapter 1, 2, 3, 7, 11, 13** ve UML sunumu (WhatsApp: *"1 2 3 7 11 13. üniteleri işledi hepsinden sorumluyuz"*). Hafta 8 videosu klasörde yok; Hafta 9'un son iki parçası (UML) hocanın *"sınavımız bugünkü konulara kadar"* dediği kısımdır ve dahildir. Hafta 10'dan sonrası (C# uygulamaları) final konusudur.
- Sınav biçimi: **4–5 klasik soru**, ezbere ve çizime dayalı (ISO tablosu, stable intermediate forms, cohesion/coupling kodu, use case / class / sequence diyagramı). Cevaplar İngilizce terimlerle yazılmalı.
- Konu dağılımı ve tahmin: `03_Soru_Analizi.pdf`.

İçindekiler

1 2025–2026 Güz Vizesi (öğrenci aktarımı, 5 soru) 2

1.1 Soru 1 – ISO/IEC 25010 (18 puan) . . . 2
1.2 Soru 2 – Stable Intermediate Forms . . . 2
1.3 Soru 3 – Cohesion / coupling türü ve düzeltilmesi . . . 3
1.4 Soru 4 – Rent a car otomasyonu: sınıf ilişkileri . . . 4
1.5 Soru 5 – Rent a car: sequence diyagramı (24 puan) . . . 5

2 2024–2025 Güz Vizesi (öğrenci aktarımı + hocanın çözümü) 6

2.1 Soru 1 – Stable Intermediate Forms (15 puan) . . . 6
2.2 Soru 2 – ISO/IEC 25010 tablosu (komple) . . . 7
2.3 Soru 3 – E-ticaret use case . . . 7
2.4 Soru 4 – Cohesion/coupling: tür ve daha iyi hâle getirme . . . 7

3 Geçmiş yıl sınavı: sınıf diyagramındaki ilişkileri yorumlama (fotoğraf, yılı belirsiz) 8

4 Online dönem test vizesi (çoktan seçmeli, docx) 9

5 2014–2015 Vize I (sınav kâğıdı fotoğrafı, Bahriye Akay, 27.11.2014) 11

5.1 Soru 1 – Unified Process (15 p) 11

5.2 Soru 2 – IEEE SPMP başlıkları (15 p) 12

5.3 Soru 3 – Cohesion ve coupling (15 p) 13

5.4 Soru 4 – Singleton pattern (15 p) 13

5.5 Soru 5 – Proje takip sistemi (40 p) 14

6 Diğer çıkmış ve ödev soruları 15

6.1 Sınıflar arası ilişki türleri 15

6.2 Adres defteri senaryosu 16

6.3 2021–22 “2. Vize Projesi”: Üniversite kütüphane platformu 17

7 Klasik çıkmış kısa sorular bankası 18

8 Alıştırmalar: koddan cohesion ve coupling bulma 22

1 2025–2026 Güz Vizesi (öğrenci aktarımı, 5 soru)

Kaynak

Soru metni yok; WhatsApp grubundaki aktarımlardan derlendi: *"1. Stable intermediate forms 2. ISO maddeleri 3. cohesion coupling çeşitleri ve düzeltilmesi 4. rent a car otomasyonu okların çeşitleri (aggregation, composition, inheritance vs) ve işlevleri 5. Otomasyonun sequence diyagramı."* Puanlar: *"İlk soru 18 puan, ana başlıklar 1'er puan, her alt başlık 0.5 puan"* (ISO sorusu), *"son soru sequence diyagramı 24 puanmış"*. Cohesion sorusundaki kod: `public void kdvekle_indirimuygula(int indirimOrani)`: indirim oranına göre toplam tutardan indirim düşüyor, sonra KDV ekliyor; öğrencilerin hatırladığı ikinci kodda `faiz = faizOrani / 100` vardı ve "tüm işlemler aynı class'ta" yapılıyordu. Aşağıdaki kodlar bu aktarımlara göre **yeniden kurulmuştur**.

1.1 Soru 1 – ISO/IEC 25010 (18 puan)

Soru 1

ISO/IEC 25010 yazılım ürün kalitesi modelinin karakteristiklerini ve alt karakteristiklerini yazınız.

Çözüm.

Ana başlık 1 puan, alt başlık 0,5 puan: 9 ana başlığın her birinden en az **iki** alt başlık yazılırsa tam puan. Güvenli olmak için hepsini yazın.

Characteristic	Sub-characteristics
Functional Suitability	functional completeness, functional correctness, functional appropriateness
Performance Efficiency	time behaviour, resource utilization, capacity
Compatibility	co-existence, interoperability
Interaction Capability	appropriateness recognizability, learnability, operability, user error protection, user engagement, inclusivity, user assistance, self-descriptiveness
Reliability	faultlessness, availability, fault tolerance, recoverability
Security	confidentiality, integrity, non-repudiation, accountability, authenticity, resistance
Maintainability	modularity, reusability, analysability, modifiability, testability
Flexibility	adaptability, scalability, installability, replaceability
Safety	operational constraint, risk identification, fail safe, hazard warning, safe integration

Açıklama istenmedi (hoca: "açıklamadan ziyade maddeler"). 2011 sürümünü bilen biri *Usability* ve *Portability* yazarsa yine puan alabilir, ama hoca 2023 tablosunu kullanıyor.

1.2 Soru 2 – Stable Intermediate Forms

Soru 2

Stable intermediate forms nelerdir? Açıklayınız.

Çözüm.

Complex systems evolve much faster and more safely if they pass through stable intermediate forms. At every step we have a stable, working form; if a change fails we return to the last stable form instead of starting again. This minimizes the effects of the moving target problem and of inevitable mistakes. Three ways:

1. **Three points of contact:** *Like a climber who keeps three limbs fixed and moves only one, we change only one thing at a time and keep everything else stable. When the change is tested*

and stable, we change the next thing. If the change has a negative effect, it is easy to go back, and we know which change caused the problem.

2. **Copy before release:** *Before a new version is released (before the change goes live), a copy/backup of the current stable version is taken, so that we can roll back if the new version fails (baselines, version control).*
3. **Iterative and incremental software development:** *The product is built in increments (mini projects) and the workflows are iterated in each increment. At the end of each increment there is a working but incomplete product, i.e. a stable intermediate form. We start with the most important core (stepwise refinement, Miller's law) and grow it like a snowball (snowball effect); risks are reduced and the product evolves with new information.*

1.3 Soru 3 – Cohesion / coupling türü ve düzeltilmesi

Soru 3 (yeniden kurulmuş kod)

Aşağıdaki modüllerin cohesion türlerini, modüller arasındaki coupling türünü bulunuz ve kodu daha iyi cohesion ve coupling'e sahip olacak şekilde yeniden yazınız.

```
class Banka
{
    public static double toplamTutar;           // global veri
    public void kdvekle_indirimuygula(int indirimOrani)
    {
        toplamTutar = toplamTutar - toplamTutar * indirimOrani / 100.0;
        toplamTutar = toplamTutar + toplamTutar * 0.20;
    }
    public void Islemler(Musteri m, double faizOrani)
    {
        double faiz = faizOrani / 100;
        Console.WriteLine("Hos geldiniz " + m.Ad);
        m.Bakiye = m.Bakiye + m.Bakiye * faiz;
        LogDosyasiniAc();
    }
}
```

Çözüm.

a) `kdvekle_indirimuygula`: communicational cohesion. *The method performs two actions (applying the discount and adding VAT). They are related by the procedure of the product (first discount, then VAT) and both operate on the same data (toplamTutar), so it has communicational cohesion.* Öğrenciler procedural, logical, temporal da yazdı; "aynı veri" ölçütü communicational'a götürür. Gerekçeyi yazmak puanı kurtarır.

b) `Islemler`: coincidental cohesion. Faiz hesaplama, karşılama mesajı yazdırma ve log dosyası açma birbiriyle ilgisiz işlerdir; aynı metotta olmaları tesadüftür. (Hepsi oturum başlangıcında art arda yapılıyorsa *temporal* demek de savunulabilir; hoca benzer örnekte "coincidental ya da temporal" dedi.)

c) Coupling:

- `toplamTutar public static`: bu alanı yazan her modül ile `kdvekle_indirimuygula` arasında **common coupling** vardır.
- `Islemler`'e bütün `Musteri` nesnesi gönderiliyor ama sadece `Ad` ve `Bakiye` kullanılıyor ⇒ çağıran ile `Islemler` arasında **stamp coupling**.

d) Düzeltilmiş kod: her metot tek iş yapar (functional), veri gizlenir ve işlemleriyle aynı sınıfta durur (informational), parametreler basit veri (data coupling).

```
class Sepet                                    // informational cohesion
{
    private double toplamTutar;                // information hiding, global yok
    public Sepet(double tutar) { toplamTutar = tutar; }
    public void IndirimUygula(int indirimOrani)          // functional
    { toplamTutar -= toplamTutar * indirimOrani / 100.0; }
    public void KdvEkle(double kdvOrani)                 // functional
    { toplamTutar += toplamTutar * kdvOrani; }
    public double Tutar() { return toplamTutar; }
}

class Hesap
{
    private double bakiye;
    public void FaizEkle(double faizOrani)               // functional, data coupling
    { bakiye += bakiye * faizOrani / 100; }
}

class Ekran   { public void Karsila(string ad) { Console.WriteLine("Hos geldiniz " + ad); } }
class Logger  { public void DosyayiAc() { /* ... */ } }
```

After refactoring each method has functional cohesion, each class has informational cohesion, and all modules are data coupled (only simple data is passed, no global data).

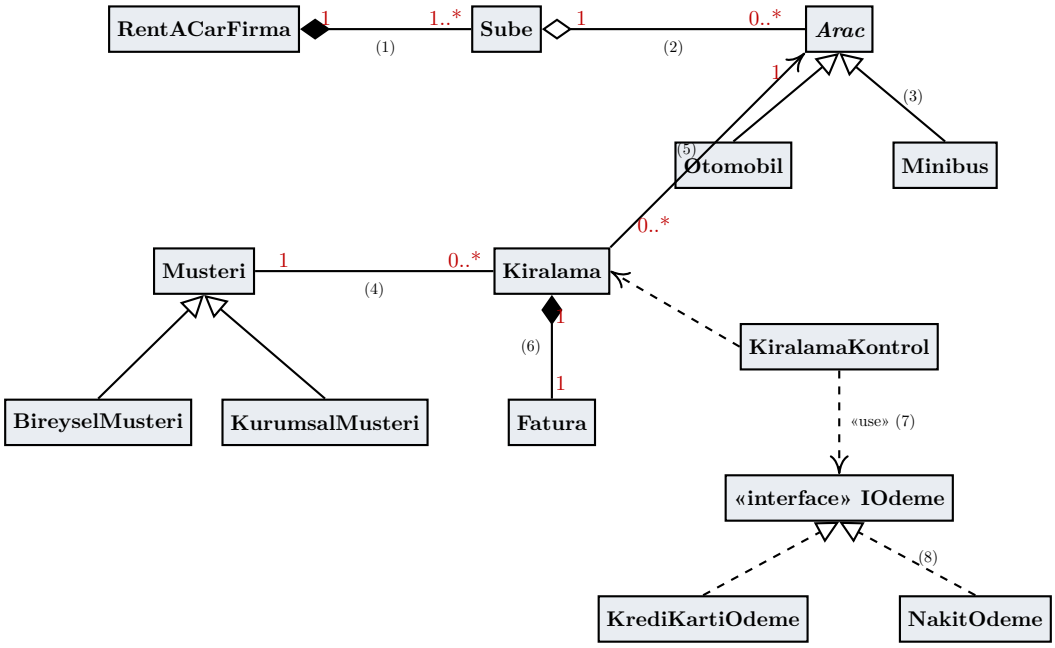
1.4 Soru 4 – Rent a car otomasyonu: sınıf ilişkileri

Soru 4 (senaryo öğrenci aktarımından)

Bir araç kiralama (rent a car) firmasının otomasyonu için sınıf diyagramı veriliyor/isteniyor. Diyagramdaki okların türlerini (association, aggregation, composition, inheritance, realization, dependency) ve senaryodaki sınıflar açısından işlevlerini açıklayınız.

Çözüm.

Sınav kâğıdındaki diyagram elimizde yok; aşağıdaki diyagram aynı senaryo için bütün ilişki türlerini içerecek şekilde kurulmuştur. Cevapta her ok için **tür + iki sınıf açısından anlam + çokluk** yazılmalıdır.



#	Relation type	Explanation
1	Composition	Firma ♦ Şube: bir firmanın bir veya daha fazla şubesi vardır; şubenin yaşam döngüsünü firma kontrol eder, firma kapanırsa şubeler de yok olur; bir şube tek bir firmaya aittir.
2	Aggregation	Şube ◊ Araç: şube bir araç topluluğu içerir (0..*); araç şubeden bağımsız yaşar, başka şubeye transfer edilebilir. Elmas **Şube** tarafında.
3	Inheritance	Otomobil ve Minibüs *is-a* Araç: plaka, marka, günlük ücret gibi ortak özellikleri Araç'tan miras alırlar. Araç italik ⇒ abstract. Bireysel/Kurumsal Müşteri de Müşteri'den kalıtılır.
4	Binary association	Müşteri – Kiralama: ikisi birbirinden haberdar; bir müşterinin sıfır veya daha fazla kiralaması olabilir, her kiralama tek müşteriye aittir.
5	Directed association	Kiralama → Araç: kiralama aracın metotlarını çağırır (`musaitMi()`, `durumGuncelle()`), araç kiralamadan haberdar değil; her kiralama tam bir araca aittir, bir araç zamanla birçok kiralamada yer alır.
6	Composition	Kiralama ♦ Fatura: fatura kiralama tarafından oluşturulur ve kiralama olmadan var olamaz (1–1).
7	Dependency	KiralamaKontrol - - «use» > IOdeme: kontrol sınıfı ödemeyi geçici olarak kullanır (parametre); ödeme arayüzü değişirse kontrol etkilenir. Aynı şekilde KiralamaKontrol Kiralama'yı kullanır.
8	Realization	KrediKartiOdeme ve NakitOdeme IOdeme arayüzünü gerçekleştirir (*can-do*); yeni ödeme türü eklemek mevcut kodu değiştirmez (Open-Closed).

Sınıf türleri: Musteri, Arac, Kiralama, Fatura, Sube **entity**; KiralamaKontrol **control**; kiralama ekranı ve fatura çıktısı **boundary** olur.

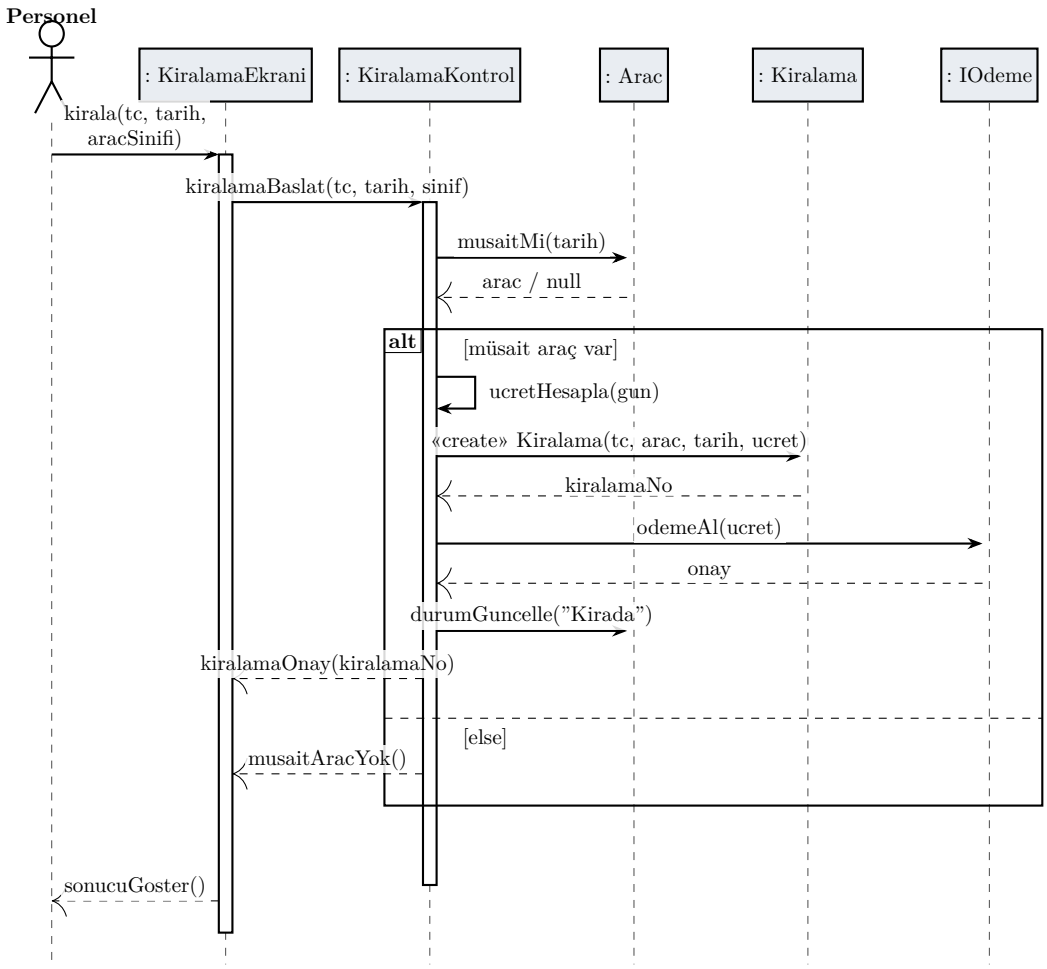
1.5 Soru 5 – Rent a car: sequence diyagramı (24 puan)

Soru 5

Rent a car otomasyonunun "Araç Kirala" senaryosu için sequence diyagramını çiziniz.

Çözüm.

Senaryo: Personel müşterinin bilgisini ve istediği tarih aralığını girer; sistem müsait araç arar; araç **varsa** ücret hesaplanır, kiralama oluşturulur, ödeme alınır, araç "kirada" olarak işaretlenir ve onay gösterilir; araç **yoksa** "müsait araç yok" mesajı gösterilir.



Puan kazandıran noktalar: (1) aktör + boundary + control + entity nesneleri doğru sırada; (2) her mesaj **okun ucundaki sınıfın metodu** ve parametreli yazılmış; (3) dönüşler kesikli; (4) aktivasyon kutuları; (5) if/else için `alt` çerçevesi; (6) self call (`ucretHesapla`); (7) akış use case adımlarıyla tutarlı.

2 2024–2025 Güz Vizesi (öğrenci aktarımı + hocanın çözümü)

Kaynak

`Çıkmışlar/2024-2025 Vize Soruları.md`, WhatsApp aktarımları (*"geçen sene vizede Stable Intermediate Forms konusunu açıklamalarıyla, ISO tablosunu komple, cohesion-coupling kodu türü ve düzeltimi bir de use case çıkmıştı"*) ve hocanın vizeden sonra tahtada gösterdiği use case çözümünün fotoğrafları.

2.1 Soru 1 – Stable Intermediate Forms (15 puan)

Soru 1

Stable intermediate forms'u açıklayınız.

Çözüm.

Cevap 2025–26 Soru 2 ile aynıdır: **three points of contact**, **copy before release**, **iterative and**

incremental software development (stepwise refinement, snowball effect), her biri bir–iki cümle İngilizce açıklamayla. Ayrıntı: Konu 2, Bölüm 2.7.

2.2 Soru 2 – ISO/IEC 25010 tablosu (komple)

Soru 2

ISO/IEC 25010 kalite karakteristiklerini ve alt karakteristiklerini yazınız.

Çözüm.

9 ana başlık ve 40 alt başlığın tamamı (2025–26 Soru 1'deki tablo). O yıl "hepsi" istendi; WhatsApp'a göre tablo 20–25 puan civarındaydı.

2.3 Soru 3 – E-ticaret use case

Soru 3

Bir alışveriş sitesinde kullanıcı ürün arayabilir/filtreleyebilir, sepete ekleyebilir, stokta varsa ödeme yaparak satın alabilir; yönetici ürün ve kullanıcıları yönetir. Use case diyagramını çiziniz ve use case'leri (brief description, pre/post condition, step-by-step, special cases) açıklayınız.

Çözüm.

Hocanın çözümü (diyagram ve *Delete Product* açıklaması) Konu 6, Bölüm 6.7'de çizilmiştir. Özet:

- Aktörler: **User**, **Admin** (gerekirse ödeme sistemi de insan olmayan aktör).
- User: Login, Register, List Products («include» Search/Filter), View Cart (sepete ekleme dahil), Payment («include» Approve Card).
- Admin: Modify Users, Add Product, Update Product, Delete Product («include» Search/Filter); Cancel ve Timeout «extend» Delete Product.

Add to Cart açıklaması (soruda vurgulanan işlem):

- **Brief description:** The user adds a selected product to the shopping cart.
- **Actor:** User. **Pre-condition:** The user is logged in and the product is listed. **Post-condition:** The product is in the cart and the cart total is updated.
- **Steps:** 1. User searches/lists products. 2. User selects a product and the quantity. 3. System checks the stock. 4. System adds the product to the cart. 5. System shows the updated cart.
- **Special cases:** product is out of stock (extend: Out of Stock); quantity exceeds stock; user cancels.

2.4 Soru 4 – Cohesion/coupling: tür ve daha iyi hâle getirme

Soru 4 (kod hatırlanmıyor; hocanın derste aynı mantıkla verdiği örnek)

```
void Yazdir(bool isDoc)
{
    if (isDoc) UsePrinter("network");
    else       UsePdfWriter(new int[] {1, 2, 3, 4, 5});
}
```

```
void UsePrinter(string b)  { Console.WriteLine(b); }
void UsePdfWriter(int[] a) { Console.WriteLine(a[0] + a[a.Length - 1]); }
void WriteTimePrint()      { Console.WriteLine(DateTime.Now); Print(true); }
```

Modüllerin cohesion türlerini ve aralarındaki coupling'i yazınız; daha iyi cohesion/coupling ile yeniden yazınız.

Çözüm.

- `UsePrinter`: **functional** (tek iş). `UsePdfWriter`: **functional**.
- `WriteTimePrint`: **coincidental** (zaman yazdırma ile yazıcıya gönderme ilgisiz) ya da **temporal** (art arda yapıldıkları için bir arada) — hoca ikisini de kabul etti.
- `Yazdir`: bayrağa göre iki ilgili işten birini seçer ⇒ **logical**; onu `isDoc` bayrağıyla çağıran modül ile arasında **control coupling**.
- `Yazdir` → `UsePdfWriter`: bütün dizi gönderiliyor, yalnızca ilk ve son eleman kullanılıyor ⇒ **stamp coupling**. `UsePrinter("network")`: basit veri ⇒ **data coupling**.

Düzeltilmiş hâli:

```
interface IYazici { void Yaz(); }                        // OCP: yeni cikti turu = yeni sinif
class AgYazici : IYazici { public void Yaz() { Console.WriteLine("network"); } }
class PdfYazici : IYazici
{
    private int ilk, son;
    public PdfYazici(int ilk, int son) { this.ilk = ilk; this.son = son; } // sadece gereken
    veri
    public void Yaz() { Console.WriteLine(ilk + son); }
}
void ZamaniYaz()   { Console.WriteLine(DateTime.Now); } // functional
void Yazdir(IYazici y) { y.Yaz(); }                      // polymorphism, bayrak yok
```

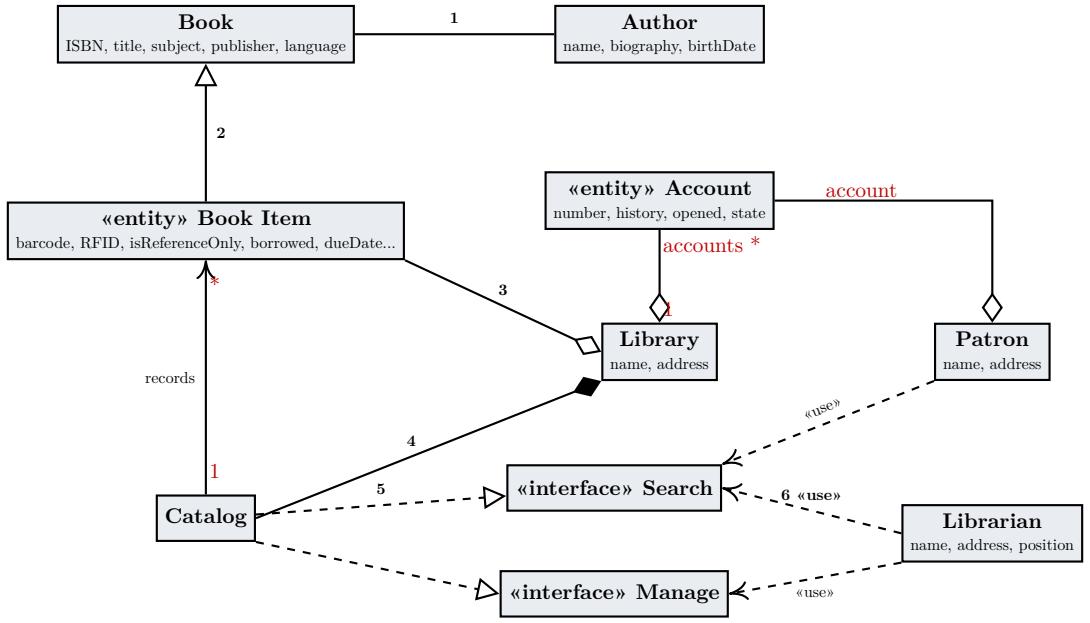
Her metot functional, sınıflar informational; bayrak kalktığı için control coupling, gereken veri gönderildiği için stamp coupling ortadan kalktı (data coupling).

3 Geçmiş yıl sınavı: sınıf diyagramındaki ilişkileri yorumlama (fotoğraf, yılı belirsiz)

Kaynak

WhatsApp grubunda (12.11.2025) paylaşılan sınav kâğıdı fotoğrafı: *"1. Aşağıdaki şekilde verilen ilişkilerin tipini ve senaryodaki sınıflar açısından ne anlama geldiğini aşağıdaki tabloya yazınız. (Her biri 5P, Toplam 25 P)"* Diyagram uml-diagrams.org'daki *Library Domain Model*'dir. Hoca 2025'te de *"class diyagramı verip aralarındaki ilişkileri yorumlamanızı isteyebilirim ama detaylı isterim"* dedi.

Library Domain Model (çizim fotoğraftan yeniden üretildi)



Numaralı ilişkilerin türünü ve sınıflar açısından anlamını tabloya yazınız.

Çözüm.

No	Relation type	Explanation
1	Association	Class Book and class Author know each other: a book is written by author(s) and an author has books; each can call the other's methods (binary association, solid line).
2	Inheritance (generalization)	Book Item *is a* Book: a book item is a physical copy of a book; it inherits ISBN, title, subject, publisher… from Book and adds its own attributes (barcode, RFID, dueDate…). The hollow triangle points to the superclass Book.
3	Aggregation	Library ◊ Book Item: the library contains a collection of book items (collection–member). A book item can exist without the library (it can be borrowed, transferred); no lifetime control. Hollow diamond on the Library side.
4	Composition	Library ♦ Catalog: the catalog is a part of exactly one library and cannot exist without it; the library creates and destroys its catalog (lifetime control). Filled diamond on the Library side.
5	Realization	Catalog - - ▷ «interface» Search: the Catalog class implements the operations declared in the Search interface (it also realizes Manage).
6	Dependency («use»)	Librarian - - > Search: the librarian uses the Search interface (e.g. to find a book); a change in Search may affect Librarian. Patron also uses Search; Librarian uses Manage.

Ek yorumlar ("detaylı isterim"): Library ◊ Account: **bir** kütüphanenin **çok sayıda** hesabı vardır; Patron ◊ Account: her okuyucunun bir hesabı vardır; Catalog → Book Item (*records*): katalog **çok sayıda** kitap kopyasını kaydeder, tek yönlüdür; «enumeration» Language, AccountState, Format ve «dataType» Address, FullName nitelik tipleri olarak kullanılır.

4 Online dönem test vizesi (çoktan seçmeli, docx)

Kaynak

`oop_ckms_sorular(test).docx` içindeki "VİZEDE ÇIKAN SORULAR" bölümü. Hoca Hafta 9 videosunda o yılın vizesi için *"10 soru var, 40 dakika olacak"* demişti. Biçim değişti (artık klasik), ama konular aynı.

Test 1

Which cohesion level does the module given below have?

```
void metot2(int opCode)
{
    if (opCode == 3)      PrintLocalSalesReport();
    else if (opCode == 4) PrintRegionalSalesReport();
    else if (opCode == 5) PrintNationalSalesReport();
    else                  PrintInternationalSalesReport();
}
```

a) Functional b) Procedural c) Communicational d) Temporal e) Logical

Çözüm.

e) Logical cohesion. Modül ilgili bir dizi eylem (dört rapor) yapar ve hangisinin çalışacağını çağıran modül `opCode` ile seçer. Çağıran ile arasında **control coupling** vardır.

Test 2

Which type of coupling is there between metot1 and metot2?

```
void metot1()
{
    int[] a = new int[10];
    for (int i = 0; i < 10; i++) a[i] = i * i;
    int s = metot2(a);
    Console.WriteLine(s);
}
int metot2(int[] dizi) { return dizi[0] + dizi[9]; }
```

a) Stamp b) Data c) Content d) Control e) Common

Çözüm.

a) Stamp coupling. Bir veri yapısı (10 elemanlı dizi) gönderiliyor ama `metot2` yalnızca `dizi[0]` ve `dizi[9]`'u kullanıyor; gereğinden fazla veri geçiyor. Hoca: *"Dizi gönderiyorsunuz ama dizinin bütün elemanları kullanılmıyor, sadece bazı elemanları; bu da stamp coupling."* Düzeltme: `int metot2(int ilk, int son)` ⇒ data coupling.

Test 3

Which one is NOT a phase of Unified Process? a) Elaboration b) Construction c) Testing d) Transition e) Inception

Çözüm.

c) Testing. Fazlar: inception, elaboration, construction, transition. Test bir *workflow*'dur.

Test 4

Which ones are the types of maintenance? (i. Perfective, ii. Corrective, iii. Adaptive, iv. Reliable) a) I and IV b) Only I c) I, II and III d) Only II e) I, III and IV

Çözüm.

c) I, II ve III. "Reliable" bir bakım türü değildir.

Test 5

Which one yields a robust architecture? a) Loose coupling / high cohesion b) High coupling / high cohesion c) Low coupling / low cohesion d) Tight coupling e) Loose coupling / low cohesion

Çözüm.

a) Loose (low) coupling / high cohesion.

Test 6

Please select the correct definition for modern maintenance. a) Development and maintenance b) Development, install and maintenance c) Maintenance occurs whenever software is modified d) Development, maintenance and development e) Development, reuse

Çözüm.

c) ISO/IEC tanımı: *maintenance occurs whenever software is modified*, kurulumdan önce ya da sonra fark etmez. (a ve b klasik *development-then-maintenance* modelidir.)

Test 7

Which one is not a separate phase during software development? a) Implementation b) Analysis c) Design d) Documentation e) Requirement

Çözüm.

d) Documentation. Dokümantasyon (testing ve planning gibi) bütün fazlarla paralel yürütülür.

Test 8

Which one is NOT an ISO/IEC 25010 quality characteristic? a) Reliability b) Portability c) Compatibility d) Comprehensibility e) Security

Çözüm.

d) Comprehensibility. (Portability 2011 sürümünde vardır; 2023'te Flexibility adını aldı.)

5 2014–2015 Vize I (sınav kâğıdı fotoğrafı, Bahriye Akay, 27.11.2014)

Kaynak

`Çıkmışlar/WhatsApp Image 2021-11-22.jpg` ve `OOP Çıkmış Sorular.pdf` sayfa 25. Eski vize notundaki *"sorarım demiyorum, soracağım diyorum"* ifadeleri bu sınavın sorularıyla örtüşüyor.

5.1 Soru 1 – Unified Process (15 p)

Soru 1

Unified Process nedir? Fazlarını ve iş akışlarını açıklayınız.

Çözüm.

The Unified Process is an object-oriented software development methodology published by Booch, Jacobson and Rumbaugh (1999). It is not a series of steps; it is an adaptable methodology modified

for each product. It is a modeling technique that uses UML, and it is iterative, incremental and two-dimensional (workflows = technical context, phases = business context).

Workflows:

- **Requirements:** determine the client's needs: understand the domain, build the business model, determine the constraints (deadline, parallel running, portability, reliability, response time, cost).
- **Analysis:** analyze and refine the requirements in a precise language for the designers; specification document; after sign-off, the SPMP (cost and duration estimates, deliverables, milestones, budget).
- **Design:** refine the analysis until it can be implemented; architectural design (modules/classes) and detailed design (data structures, algorithms); choose language, reuse, portability.
- **Implementation:** implement the product in the chosen language (subsystems, components).
- **Test:** responsibility of every developer and the SQA group; traceability; reviews; unit, integration, product and acceptance testing; alpha and beta releases.

Phases:

- **Inception:** is the product economically viable? Understand the domain, build the business model, delimit the scope, initial business case; identify the three risk categories (technical, requirements, architecture). Deliverables: initial domain/business/requirements models, initial risk list, plan for elaboration, initial business case.
- **Elaboration:** refine the requirements and architecture, monitor risks, refine the business case, produce the project management plan.
- **Construction:** produce the first operational-quality version (beta release); emphasis on implementation and testing; user manuals.
- **Transition:** ensure the client's requirements are met; correct faults, complete the manuals, discover unidentified risks; driven by feedback from beta sites.

5.2 Soru 2 – IEEE SPMP başlıkları (15 p)

Soru 2

IEEE'nin yazılım proje yönetim planının başlıklarını yazınız.

Çözüm.

Hoca yalnızca 8 ana başlığı ister:

1. **Overview** (project summary: purpose, scope and objectives; assumptions and constraints; deliverables; schedule and budget summary; evolution of the plan)
2. **Reference materials**
3. **Definitions and acronyms**
4. **Project organization** (external interfaces, internal structure, roles and responsibilities)
5. **Managerial process plans** (start-up, work, control, risk management, project close-out plans)
6. **Technical process plans** (process model; methods, tools and techniques; infrastructure; product acceptance)

7. **Supporting process plans** (configuration management, testing, documentation, quality assurance, reviews and audits, problem resolution, subcontractor management, process improvement)
8. **Additional plans**

5.3 Soru 3 – Cohesion ve coupling (15 p)

Soru 3

Kohezyon ve coupling'i açıklayınız. Türlerinin başlıklarını yazınız.

Çözüm.

Cohesion is the degree of interaction within a module; coupling is the degree of interaction between two modules. A good design has maximal interaction within a module (high cohesion) and minimal interaction between modules (low coupling); this makes maintenance easier and reduces regression faults.

Cohesion (kötüden iyiye)	Coupling (kötüden iyiye)
1. Coincidental	1. Content
2. Logical	2. Common
3. Temporal	3. Control
4. Procedural	4. Stamp
5. Communicational	5. Data
6. Functional	
7. Informational	

Her birine birer cümlelik tanım eklemek (Konu 4) puanı garantiler.

5.4 Soru 4 – Singleton pattern (15 p)

Soru 4

a) Singleton pattern'i açıklayınız. b) Çözümünü bulunuz. c) Kodla örnek veriniz.

Çözüm.

Not

Bugün design pattern'ler vizeden sonra işleniyor (final konusu). 2014'te vizede sorulmuştu; kısa cevabı bilmekte fayda var.

a) *Singleton guarantees that a class has only one instance during the life cycle of the program and provides a global point of access to it.* b) Constructor `private` yapılır, tek nesne sınıfın içinde `static` bir alanda tutulur, erişim `static` bir metot/property üzerinden verilir; sınıf `sealed` yapılarak kalıtım engellenir.

```
public sealed class Ayarlar
{
    private static Ayarlar nesne;                // tek ornek
    private Ayarlar() { }                        // disaridan new yapilamaz
    public static Ayarlar Nesne
    {
        get
```

```
        {
            if (nesne == null) nesne = new Ayarlar();
            return nesne;
        }
    }
}
// kullanim: Ayarlar a = Ayarlar.Nesne;
```

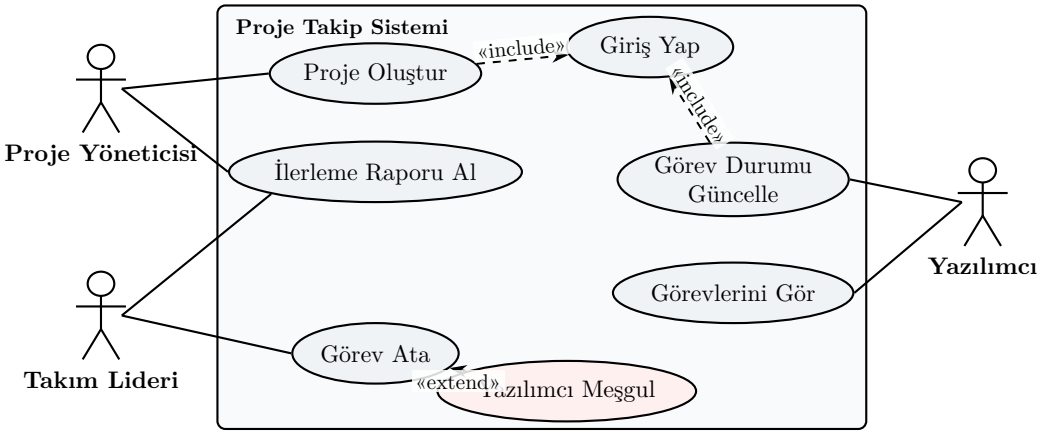
5.5 Soru 5 – Proje takip sistemi (40 p)

Soru 5

Proje yöneticisi, takım lideri ve yazılımcıların olduğu bir proje takip sistemi tasarlanacaktır. a) Use case'lerini oluşturunuz (10). b) Class yapısını oluşturun, class tiplerini (entity, boundary, control) belirtin (10). c) Class'ların tasarımını oluşturunuz (10). d) Collaboration veya sequence diyagramlarından birini seçip oluşturunuz (10).

Çözüm.

a) Use case diyagramı



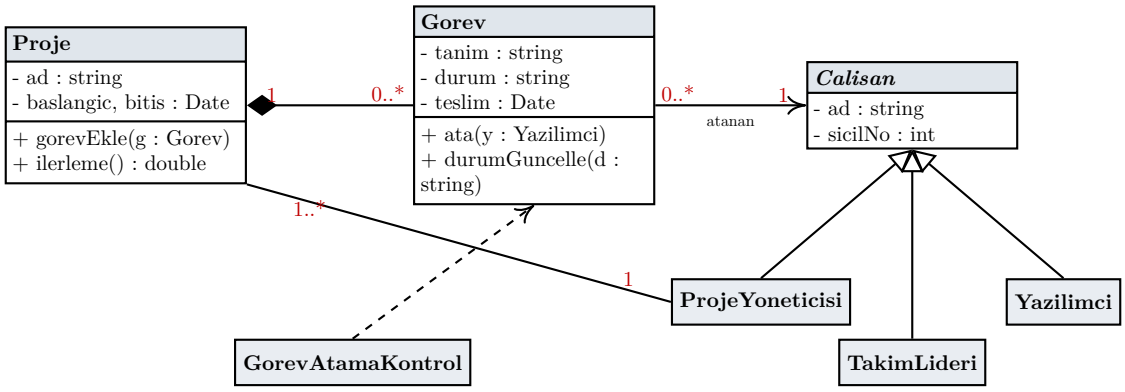
Takım Lideri de ilerleme raporu alabilir. İstenirse ortak bir *Çalışan* aktörü tanımlanıp üç rol ondan **generalization** ile özelleştirilebilir (boş üçgen genel aktöre doğru).

Görev Ata açıklaması: brief: the team leader assigns a task of a project to a developer; actor: Team Leader; pre: the project and the developer exist; post: the task is recorded with its developer and deadline; steps: 1. leader selects the project, 2. system lists the tasks and the developers, 3. leader selects a task and a developer and enters a deadline, 4. system saves the assignment and notifies the developer; special cases: developer busy (extend), leader cancels.

b) Sınıflar ve türleri

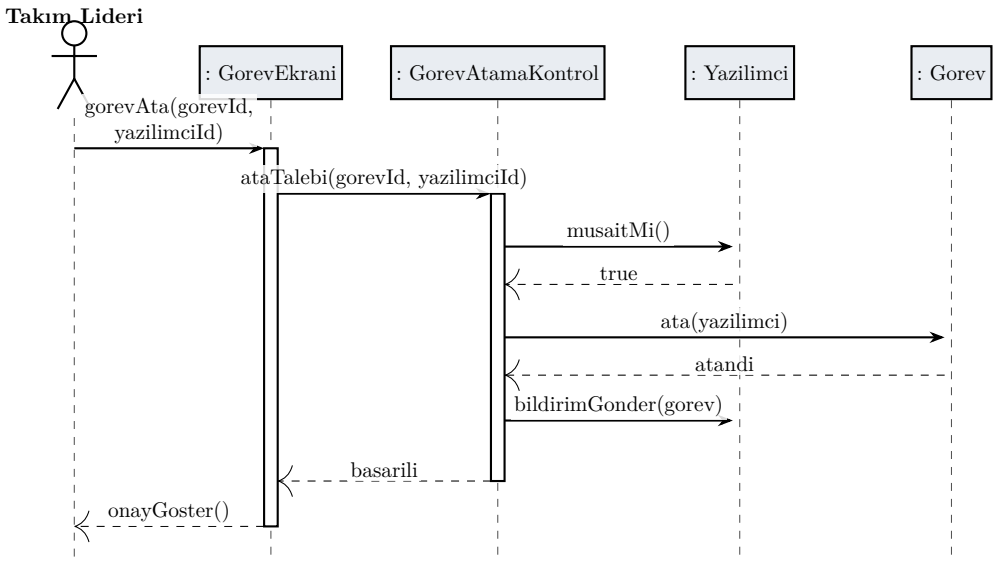
Sınıf	Tür	Gerekçe
Proje, Gorev, Calisan (ProjeYoneticisi, TakimLideri, Yazilimci)	Entity	Uzun ömürlü bilgi tutar.
ProjeEkrani, GorevEkrani, IlerlemeRaporu	Boundary	Giriş/çıkış ekranları ve rapor.
GorevAtamaKontrol, IlerlemeHesaplayici	Control	Atama kuralları ve ilerleme yüzdesi hesabı.

c) Sınıf tasarımı



Proje ♦ Görev composition (görev projesiz yaşamaz); Görev → Çalışan yönlü ilişki (atanan); ProjeYöneticisi, TakımLideri, Yazılımcı Çalışan'dan kalıtılır; kontrol sınıfı görevleri kullanır (dependency).

d) Sequence diyagramı (Görev Ata)



6 Diğer çıkmış ve ödev soruları

6.1 Sınıflar arası ilişki türleri

Kaynak

`Notlar/oop özet.docx` ("OOP 1.) SORU" cevabı) ve WhatsApp'ta (12.11.2025) paylaşılan "OOP 1.) SORU" görseli: altı ilişki çizgisi (Association, Inheritance, Realization, Dependency, Aggregation, Composition) verilip açıklaması istenmiş. Yılı belirsiz.

Soru

Aşağıdaki sınıf ilişkilerini açıklayınız, gösterimlerini çiziniz: a) Association (unary ve binary) b) Inheritance c) Realization d) Dependency e) Aggregation f) Composition.

Çözüm.

a) Association *Unary:* class A knows about class B but B knows nothing about A; drawn as a line with an open arrowhead pointing to B (the called class). *Binary:* A knows about B and B knows about A (they call each other's methods); drawn as a plain solid line.

b) Inheritance (generalization) "is-a": the subclass inherits the attributes and operations of the superclass; solid line with a **hollow triangle** pointing to the superclass.

c) Realization A class implements an interface ("can-do"); the interface contains only method names and parameters. **Dashed line with a hollow triangle** pointing to the interface. (Not: öğrenci cevabında "içi dolu ok" yazıyor; bazı çizim araçları üçgeni renkli doldurur ama UML standardında üçgen **boştur**.)

d) Dependency "uses-a": when the definition of one element changes, the other may be affected (parameter, local variable); **dashed line with an open arrowhead**.

e) Aggregation "has-a" / collection–member; the part can exist without the whole; **hollow diamond on the whole (collection) side**.

f) Composition The strongest relationship; part–whole with lifetime control, the part cannot exist without the whole; **filled diamond on the whole side**.

Çizimleri ve C# karşılıkları: Konu 8, Bölüm 8.2.2 ve 8.2.3.

6.2 Adres defteri senaryosu

Kaynak

`Çıkmışlar/OOP.docx`: senaryo ve use case açıklama alanları (use-case adı, brief description, entry/exit condition, step-by-step, exceptional cases). Hoca aynı örneği derste use case + sınıf + CRC + collaboration ile tam çözdü.

Soru

Kişilere ait ad, soyad, adres ve telefon bilgisinin tutulduğu bir adres defteri tanımlanmak isteniyor. Adres defterine kişi ekleme, eklenmiş kişide güncelleme yapabilme, silme ve bulma (arama) işlemleri olacaktır. Ayrıca kişileri alfabetik olarak listeleyen bir işlev de gereklidir. Use case'leri (use-case adı, brief description, entry condition, exit condition, step by step description, exceptional cases) ile birlikte modelleyiniz; sınıfları, CRC kartlarını ve bir collaboration/sequence diyagramını çiziniz.

Çözüm.

- Use case diyagramı ve *Kişi Ekle*, *Kişi Güncelle*, *Listele* açıklamaları: **Konu 6, Bölüm 6.6**.
- Sınıf diyagramı (Kisi entity, AdresDefteri, Menu boundary, Program), CRC kartları ve *Kişi Ekle* collaboration + sequence diyagramı: **Konu 7, Bölüm 7.7** (hocanın kendi çözümü).
- **Kişi Sil** açıklaması (eksik kalmasın): brief: bulunan bir kişinin defterden silinmesi; entry: silinecek kişinin adı ve soyadı bilinmeli; exit: kişi defterden silinmiş; steps: 1. kullanıcı silmeyi seçer, 2. kişinin adı soyadı girilir, 3. sistem kişiyi arar (include Arama), 4. kişinin detayları gösterilir, 5. kullanıcı onaylar, 6. sistem kişiyi siler; exceptional: kişi yok, kullanıcı vazgeçti.
- **Kişi Ara** açıklaması: brief: ad ve soyada göre kişinin bulunması; entry: defterde en az bir kişi; exit: kişinin bilgileri gösterilir; steps: 1. ad soyad girilir, 2. sistem kişileri tarar, 3. bulunan kişinin adres ve telefonu gösterilir; exceptional: kişi yok.

6.3 2021–22 "2. Vize Projesi": Üniversite kütüphane platformu

Kaynak

`Notlar/Project.pdf` (Prof. Dr. Bahriye Akay, 06.12.2021). O yıl ikinci vizenin yerine geçen ödevdi; bugün sınavda gelebilecek senaryo sorularının tam bir örneğidir ve hocanın puanlamasını gösterir: gereksinim iş akışı 25 P (alan anlatımı, sözlük, use case + kısa tanım + adım adım tanım, işlevsel/işlevsel olmayan gereksinimler), analiz iş akışı 55 P (entity/boundary/control sınıfları, CRC ve class diyagramı, statechart, her senaryo için sequence, IEEE SPMP, test planı), tasarım 25 P.

Senaryo

Sistem yöneticileri kitap tanımlar, kitapları öğretim üyelerine ve öğrencilere ödünç verir, iade alır, bu kişilerin bilgilerini düzenler. Öğrenciler ve öğretim üyeleri kitap listesinden kitap seçer, kitap alma talebinde bulunur ve okuduktan sonra iade eder. Gereksinim ve analiz iş akışlarını modelleyiniz.

Çözüm.

Sözlük (glossary) örneği: *Ödünç (loan):* bir kitap kopyasının belirli bir süre için bir üyeye verilmesi. *Üye (member):* öğrenci veya öğretim üyesi. *İade tarihi (due date):* kitabın geri getirilmesi gereken son gün. *Talep (request):* üyenin bir kitabı almak için yaptığı başvuru.

Aktörler: Sistem Yöneticisi; Üye (genel aktör) ← Öğrenci, Öğretim Üyesi (generalization, boş üçgen genele doğru).

Use case'ler: Kitap Tanımla, Üye Bilgisi Düzenle, Ödünç Ver, İade Al (yönetici); Kitap Listele («include» Kitap Ara), Kitap Talep Et (üye); Giriş Yap (hepsi «include»); istisnalar: *Kitap Mevcut Değil* «extend» Kitap Talep Et, *Gecikme* «extend» İade Al.

Gereksinimler: *Functional:* kitap ekleme/silme/düzenleme, üye düzenleme, ödünç verme, iade alma, kitap arama/listeleme, talep. *Nonfunctional:* arama 2 saniyeden kısa sürmeli; aynı anda 200 kullanıcı; web tabanlı çalışmalı; üye bilgileri yetkisiz erişime kapalı (security); haftanın 7 günü erişilebilir (availability).

Sınıflar: Entity: Kitap, KitapKopyasi, Uye (Ogrenci, OgretimUyesi), Odunc, Talep. Boundary: GirisEkrani, KitapListesiEkrani, YoneticiEkrani, GecikmeRaporu. Control: OduncKontrol (ödünç verme kuralları, gecikme hesabı), AramaKontrol.

CRC örneği:

CLASS: OduncKontrol
Responsibility: send message to KitapKopyasi to check if it is available; create an Odunc object; send message to KitapKopyasi to set its state to "loaned"; compute the due date; compute the delay on return
Collaboration: KitapKopyasi, Uye, Odunc, Talep

İlişkiler: Kitap ← KitapKopyasi (bir kitabın birden çok kopyası: Kitap ◊ 1–1..* KitapKopyasi), Uye 1 — 0..* Odunc, Odunc * → 1 KitapKopyasi, Ogrenci/OgretimUyesi → Uye (inheritance).

Statechart (KitapKopyasi): *Rafta* –[ödünç verildi]–> *Ödünçte* –[iade edildi]–> *Rafta*; *Ödünçte* –[süre doldu]–> *Gecikmede* –[iade edildi]–> *Rafta*; *Rafta* –[talep edildi]–> *Ayrıldı* –[ödünç verildi]–> *Ödünçte*.

SPMP: 8 ana başlık (Konu 3). Sequence diyagramı için Kitap Talep Et senaryosu, rent-a-car çözümündeki şablonla çizilir (aktör → ekran → OduncKontrol → KitapKopyasi / Odunc, `alt` [mevcut]

/ [değil]).

7 Klasik çıkmış kısa sorular bankası

Kaynak

`NESNE YÖNELİMLİ PROGRAMLAMA ÇIKMIŞ SORULAR.docx` (aynı içerik `notlar2.docx` ve el yazısı `OOP Çıkmış Sorular.pdf` sayfa 1–19'da) ve eski "OOP Vize Notlar". Yalnızca vize kapsamındaki sorular alındı; cevaplar kitaba göre düzeltilerek İngilizce/Türkçe verildi.

K1

Yazılım geliştirme yaşam döngüsünün fazlarını yazınız.

Çözüm.

Klasik: requirements, analysis (specification), design, implementation, postdelivery maintenance, retirement. Nesne yönelimli iş akışları: requirements, (OO) analysis, (OO) design, implementation, test (+ postdelivery maintenance, retirement). Planning, testing ve documentation ayrı faz değildir.

K2

Regression fault nedir?

Çözüm.

A fault in an apparently unrelated part of the software caused by a change made elsewhere. Information hiding ve düşük coupling regresyon hatasını azaltır; regression testing ile yakalanır.

K3

Verification ve validation nedir?

Çözüm.

Verification: her fazın sonunda yapılan test ("ürünü doğru yapıyor muyuz?"). Validation: projenin sonunda ürünün spesifikasyonu karşılayıp karşılamadığının testi ("doğru ürünü mü yaptık?"). Kitap: ikisi de çok geç; test sürekli yapılmalı.

K4

Moving target problem nedir? Çözümü var mıdır?

Çözüm.

A change in the requirements while the software product is being developed. Bağımlılıklara ve regression fault'lara yol açar; çok değişiklik olursa ürün yeniden tasarlanır. **Çözümü yoktur**; iterative-incremental model ve robust mimari etkisini azaltır.

K5

Yineleme (iteration) ve artırım (incrementation) nedir? Miller kuralı ile açıklayınız; yararları nelerdir?

Çözüm.

Miller: insan aynı anda yaklaşık 7 bilgi parçasına odaklanabilir ⇒ stepwise refinement: en önemli kısımlar önce, diğerleri sonra (**incrementation**: ürünü parça parça büyütmek). **Iteration**: her ardışık sürüm hedefe öncekinden yakındır. Yararları: çok sayıda test fırsatı, hatalar erken bulunur; mimarinin sağlamlığı erken belli olur; riskler erken azaltılır; baştan çalışan sürüm olur, müşteri geri bildirimi erken gelir.

K6

Alfa ve beta testleri nelerdir? Sunarken nelere dikkat edilmeli?

Çözüm.

Alpha release: COTS yazılımın geliştirme testleri sürerken olası müşterilere test için verilen sürüm. Beta release: construction fazı sonunda üretilen ilk işletme kalitesindeki sürüm; transition fazı beta sitelerinden gelen geri bildirimle yürür. Alfayı çok erken verip kullanıcının gözünden düşmemeli; betada rakiplerin gerisinde kalmamak için geç kalmamalı.

K7

Riskler kaç kategoriye ayrılır?

Çözüm.

Üç: **technical risks**, **the risk of not getting the requirements right**, **the risk of not getting the architecture right**. Riskler sıralanır, kritik olanlar önce azaltılır.

K8

Inception fazının adımları nelerdir?

Çözüm.

1) Understand the domain, 2) build the business model, 3) delimit the scope of the project, 4) begin to make the initial business case (maliyet-etkinlik, yatırımın geri dönüşü, pazar, zamanında teslim, riskler). Amaç: ürün ekonomik olarak yapılabilir mi?

K9

Elaboration, construction ve transition fazlarını açıklayınız.

Çözüm.

Elaboration: başlangıç çıktılarını incelt (gereksinimler, mimari), riskleri izle, business case'i incelt, **project management plan**'ı hazırla. Construction: ilk işletme kalitesinde sürüm (**beta**); implementation, unit/integration/product testing; kullanıcı kılavuzları. Transition: müşteri gereksinimleri karşılanıyor mu, hatalar düzeltilir, kılavuzlar tamamlanır, yeni riskler aranır.

K10

İki boyutlu modele neden ihtiyaç duyulur?

Çözüm.

Gerçek projeler tek seferde tamamlanamayacak kadar büyüktür; Miller kuralı gereği artırımlara (fazlara) bölünür ve her artırımda iş akışları yinelenir. İki boyutlu model hem iş akışlarını (teknik bağlam) hem fazları (iş bağlamı) gösterir; moving target ve kaçınılmaz hatalarla daha iyi başa çıkar, hatalar daha kolay görülür.

K11

Analiz iş akışındaki modeller (modelleme türleri) nelerdir?

Çözüm.

Functional modeling (senaryolar), **entity class modeling** (sınıflar, nitelikler, ilişkiler: class diagram), **dynamic modeling** (her sınıfa yapılan işlemler, durum geçişleri: statechart).

K12

CRC kartı nedir, ne işe yarar, nasıl gösterilir?

Çözüm.

Class–Responsibility–Collaboration: her sınıf için adı, sorumlulukları (ne yapar/bilir) ve işbirliği yaptığı sınıflar yazılır. Sınıfları bulmak (alan bilgisi gerekir) ve özellikle analiz artifact'larını **test** etmek için kullanılır; ekip kartları canlandırınca eksik/hatalı öğeler ortaya çıkar.

K13

Use case nedir? Include ve extends neyi ifade eder?

Çözüm.

Use case, ürün ile aktörleri arasındaki etkileşimi modeller; sistem dikdörtgen, aktör çöp adam, use case elips. «include»: ana use case'in her seferinde kullandığı, yeniden kullanım için ayrılmış davranış (ok ana → kullanılan). «extend»: istisnai/nadir durumlar (ok istisna → ana).

K14

Distribute the control (dağıtık kontrol) nedir?

Çözüm.

Bütün sorumluluğu tek sınıfa yüklemek (God class) yerine kontrolü dağıtmak: asansör probleminde her asansöre ve her kata bir alt kontrolcü ve bir zamanlayıcı (scheduler).

K15

Collaboration diagram nedir?

Çözüm.

Bir senaryonun gerçekleştirilmesinde nesneleri, aralarındaki bağlantıları ve **gönderilme sırasına göre numaralanmış** mesajları gösteren interaction diagram; eşdeğeri sequence diagram'dır.

K16

Specification document nedir, önemi nedir?

Çözüm.

Müşteri ile geliştirici arasında **sözleşmedir**; kesin olmalı, çelişki, eksiklik ve tamamlanmamışlık içermemeli. Test ve bakım için şarttır. Unified Process'te use case'ler ve onlardan türeyen UML artifact'ları bu rolü oynar (müşteri için metin, tasarımcı için diyagram).

K17

Gereksinimler kaç grupta toplanır?

Çözüm.

Functional (ürünün yapması gereken eylemler; girdi–çıktı) ve **nonfunctional** (ürünün özellikleri: platform, yanıt süresi, güvenilirlik; bazıları tasarıma kadar bekler).

K18

CASE aracı nedir? Örnek veriniz.

Çözüm.

Computer-Aided Software Engineering: UML diyagramlarını çizmek, değiştirmek, belgeleri saklamak ve süreci izlemek için yazılım araçları. IBM Rational Rose, Together, ArgoUML, System Architect.

K19

Sınıf türlerini yazarak açıklayınız.

Çözüm.

Entity (uzun ömürlü bilgi), boundary (ürün–çevre etkileşimi: ekran, rapor), control (karmaşık hesaplama ve algoritmalar). Ayrıntı: Konu 7.

K20

Information hiding nedir, neden ihtiyaç duyulur?

Çözüm.

Değişmesi muhtemel ayrıntıların modülün içinde gizlenmesi; değişiklik yerelleşir, diğer modüller etkilenmez, regression azalır; kullanıcı kritik verileri doğrudan değiştiremez (en az yetki).

K21

Abstract data type ve abstraction nedir?

Çözüm.

ADT: veri tipi + o tipin örnekleri üzerindeki işlemler (informational cohesion; sınıflar). Abstraction: gereksiz ayrıntıları gizleyip ilgili ayrıntıları vurgulayarak problemi daha üst düzeyde düşünmek (stepwise refinement aracı).

K22

Modül nedir? Bir modülün kaç özelliği vardır?

Çözüm.

A lexically contiguous sequence of program statements, bounded by boundary elements, with an aggregate identifier. Üç özelliği: **function** (ne yapar), **logic** (nasıl yapar), **context** (ne üzerinde çalışır).

K23

Nesne yönelimli programlamanın bakımda sağladığı avantajlar nelerdir?

Çözüm.

Information hiding sayesinde değişiklikler yerel kalır, regression azalır; bağımsız birimler (encapsulation) hatanın yerini bulmayı kolaylaştırır; yeniden kullanım; yüksek cohesion ve düşük coupling'li modüller.

K24

Dynamic binding nedir?

Çözüm.

Hangi metodun çalışacağının nesnenin sınıfına göre **çalışma zamanında** belirlenmesi (`myFile.open()` örneği); polymorphism ile birlikte gelir. Güçlü yandır ama kodu anlamayı zorlaştırabilir.

K25

Görüşme türleri nelerdir? Görüşmeci nasıl olmalı?

Çözüm.

Structured (önceden planlanmış, kapalı uçlu sorular) ve unstructured (cevaplara göre sorulan, açık uçlu sorular). Görüşmeci alanı iyi bilmeli, açık fikirli olmalı; görüşmeden sonra yazılı rapor hazırlayıp bir kopyasını görüşülen kişiye vermeli.

8 Alıştırmalar: koddan cohesion ve coupling bulma

Neden bu bölüm?

Son iki vizede kod verilip tür ve düzeltme istendi, WhatsApp'ta da "kod örneği var mı" diye çok soruldu. Aşağıdaki kodlar sınav tarzındadır.

A1

```
void SistemKapat()
{
    log.Kapat();
    ekran.Temizle();
    yedek.Al();
}
```

Çözüm.

Temporal cohesion: işlemler birbirinden bağımsız, yalnızca kapanış anında art arda yapıldıkları için bir aradadır. Düzeltme: her iş kendi sınıfında (`Logger.Kapat()`, `Ekran.Temizle()`, `Yedekleyici.Al()`); kapanışta çağıran bunları ayrı ayrı çağırır.

A2

```
void Hesapla(char islem, double a, double b)
{
    if (islem == '+') Console.WriteLine(a + b);
    else if (islem == '-') Console.WriteLine(a - b);
    else if (islem == '*') Console.WriteLine(a * b);
}
```

Çözüm.

Logical cohesion; çağıranla **control coupling** (`islem` bir kontrol öğesi). Open-Closed ilkesi ihlali (yeni işlem için `if` eklenir). Düzeltme: `double Topla(double a, double b)`, `Cikar`, `Carp` ayrı metotlar (functional, data coupling) ya da `IIslem` arayüzü ve her işlem için sınıf. (2025–26 *finalinde* aynı tip soru SOLID ile birlikte geldi.)

A3

```
class Ogrenci { public string No, Ad, Soyad, Sifre; public double Ortalama; }
bool BursAlirMi(Ogrenci o) { return o.Ortalama >= 3.0; }
```

Çözüm.

`BursAlirMi` **functional**; çağıranla arasında **stamp coupling** (bütün kayıt, şifre dahil, gönderiliyor; yalnızca `Ortalama` kullanılıyor). Düzeltme: `bool BursAlirMi(double ortalama)` ⇒ data coupling.

A4

```
static class Durum { public static bool indirimVar; }
void KampanyaBaslat() { Durum.indirimVar = true; }
double FiyatHesapla(double f) { return Durum.indirimVar ? f * 0.9 : f; }
```

Çözüm.

İki metot arasında **common coupling** (aynı global veriyi yazıp okuyorlar). Düzeltme: `double FiyatHesapla(double f, double indirimOrani)` (data coupling) ya da kampanya bilgisini ve fiyat hesabını bir `Kampanya` sınıfında kapsülle (information hiding).

A5

```
void NotGirVeOrtalamaGuncelle(Ogrenci o, int not)
{
    o.Notlar.Add(not);
    o.Ortalama = o.Notlar.Average();
}
```

Çözüm.

Communicational cohesion: iki iş (not ekleme, ortalama güncelleme), prosedürle ilişkili ve aynı veri (`o`) üzerinde. Daha iyisi: `Ogrenci` sınıfına `NotEkle(int)` ve `Ortalama()` metotları (sınıf informational, metotlar functional).

A6

```
class Kuyruk
{
    private int[] d = new int[100]; private int bas, son;
    public void Ekle(int x) { d[son++] = x; }
    public int Cikar() { return d[bas++]; }
    public bool BosMu() { return bas == son; }
}
```

Çözüm.

Informational cohesion: her işlemin kendi giriş noktası ve bağımsız kodu var, hepsi aynı (gizli) veri yapısı üzerinde; bir ADT örneği. Düzeltme gerekmez.